

LC 980: Unique Paths - 3

Amazon Adobe

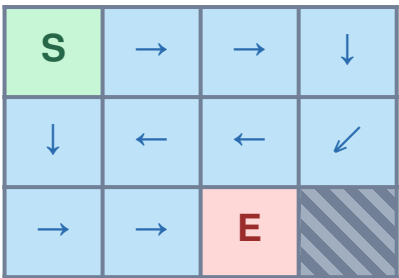
Authored by: Arpit Raj, LNMIIT Jaipur
Date: July 28, 2026

Problem Breakdown

Value	Meaning
1	start pos
2	end pos
-1	obstacle
O/P	number of ways need to traverse every walkable path from start to end cant miss any one

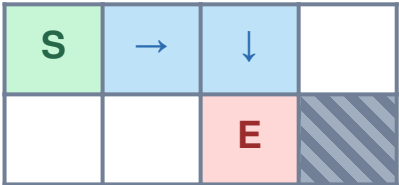
Definition: a valid path = all 0 + start + end (exactly once)

Visualizing Paths



So Eg:

2 possible ways where we are traversing all floor



Note:

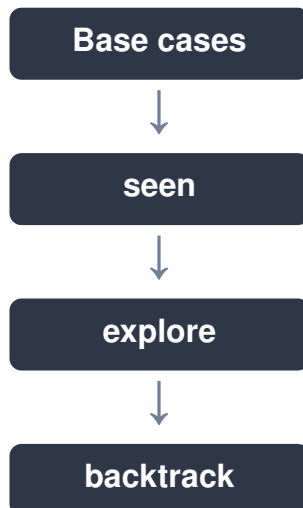
not allowed because we are not covering whole floor

| DFS Backtrack Template

We stop, only when destination and every walkable square is visited.

dfs state $\rightarrow$ (r, c, remaining)
(remaining $\rightarrow$ cells yet to visit)

Standard DFS Backtrack Template



Why backtrack?

we need to backtrack because its possible we reach destination too early via that path leaving the squares

| C++ Implementation

```

class Solution {
public:
    int n;
    int m;
    int ans;
    vector<pair<int,int>> dir={{-1,0},{0,-1},{1,0},{0,1}};

    void dfs(int r, int c, vector<vector<int>>& grid, int rem) {

        if(r>=n || r<0 || c>=m || c<0) return;

        if(grid[r][c]==2 && rem==1) {
            ans++;
            //last walkable cell should be dist to make a traversal valid
        }

        //SEB

        int temp = grid[r][c];
        grid[r][c] = -1; //marked seen

        for(auto &it:dir) { //explore
            int nr = r+it.first;
            int nc = c+it.second;
            if(nr>=0 && nc>=0 && nr<n && nc<m) {
                if(grid[nr][nc]!=-1) {
                    dfs(nr, nc, grid, rem-1);
                }
            }
        }

        grid[r][c] = temp;
    }

    int uniquePathsIII(vector<vector<int>>& grid) {
        //basic idea bas yahi he ki
        //sr , sc find krke waha se dfs chaalu krenge in attempt to find all wal
        //destination pe phuchte hue make sure krenge ki last remaining cell ho

        n = grid.size();
        m = grid[0].size();
        int rem = 0;
    }
}

```

```
int sr;
int sc;

for(int i=0; i<n; i++) {
    for(int j=0; j<m; j++) {

        if(grid[i][j]!=-1) {
            rem++; //walkable cells
        }
        if(grid[i][j]==1) {
            sr=i;
            sc=j;
        }
    }
}

dfs(sr, sc, grid, rem);
return ans;
}
};
```